

Comprehensive Exam (Open-book) Part B

Date: 22th Feb 2021

Max. Marks: 60[Part B]

Duration: Max. 1 hr.30 min

Important notes:

1. This is PART B worth 60 marks, which should be attempted after Part A. Max. time is 1hr 30 min.
 2. Write your answers in the Google Form provided for Part B.
 3. The test is OPEN BOOK, and you can access your notes, books, printout of slides, or any other material in hard copy. Any material in soft copy or any other resource online is PROHIBITED (including the use of compilers). Use of calculators or mobile phones is also prohibited.
1. Complete the following code which prints the pattern (depicted on the right) in a file called "pattern_file". The argument required for the pattern is passed as a command line argument. Eg: **./a.out 5**
Assume you have all required header files already available. However, you have to complete the code without the use of any loops (for or while) or branching statements (if/else). **[15M]**

```
FILE *fp; //this is a global variable
```

```
void pattern(int n, int i, FILE* f) {
```

```
    if (n < 1)
```

```
        return;
```

```
    if (i <= n) {
```

```
        // (d) your code here in not
        // more than two lines
```

```
    }
```

```
    else {
```

```
        // (e) your code here in not
        // more than two lines
```

```
    }
```

```
}
```

```
int main(int argc, char* argv[]) {
```

```
    fp = _____ // (a) open pattern_file in write mode
```

```
    int n = _____ // (b) obtain the argument passed through command line here
```

```
    _____ // (c) make the function call here using n (and other arguments, as suitable)
```

```
    return 0;
```

```
}
```

```
pratik@Prithvi:~/code$ ./a.out 5
pratik@Prithvi:~/code$ cat pattern_file
54321
4321
321
21
1
pratik@Prithvi:~/code$ ./a.out 3
pratik@Prithvi:~/code$ cat pattern_file
321
21
1
```

2. An electronic company manufactures five different items. The company, as a record keeping practice, maintains the number of items manufactured and sold on a weekly basis every month. Based on the records, the production, sales, and future goals are revised. Here, being the programmer, we consider **two** two-dimensional arrays, `production[rows][cols]` and `sales[rows][cols]`, to represent production and sales of items as per weeks, where `rows` and `cols` depict the number of weeks and items, respectively. For example, the array entry `production[0][0]` represents the production of item 0 in week 0, `production[0][1]` for production of item 1 in week 0, `production[1][0]` for production of item 0 in week 1, up to `production[2][4]` for production of item 2 in week 4, giving a total production count of five items in three weeks (indexing starting from zero). Similarly, the other array, `sales`, can be considered for sale of each item with respect to the corresponding weeks. Let us consider the cost of each product being stored in a one-dimensional array `cost`, with each cell storing the cost for each respective item. That is, `cost[0]` stores the cost of item 0, `cost[1]` for item 1, up to `cost[4]` for item 4.

The below program populates the production and sale details, along with the cost of each item by taking user input for production, sales and cost arrays, and uses the input data to perform some assessment. Based on the assessment, the future goals of the company are revised.

```
1.  #include <stdio.h>
2.  #define rows 3
3.  #define cols 5
4.  void weekly_production_sales(int p[rows][cols], int q[rows][cols]);
5.  void display_item_wise(int p[rows][cols], int q[rows][cols]);
6.  int production[rows][cols], sales[rows][cols], cost[cols], p_week[rows],
   s_week[rows], product_wise[cols], sale_wise[cols], pt, st, i, j;
7.  void main(){
8.      int p[rows][cols], s[rows][cols];
9.      for(i=0; i<rows; i++)
10.         for(j=0; j<cols; j++){
11.             scanf("%d", &production[i][j]);
12.             scanf("%d", &sales[i][j]);
13.         }
14.         for(i=0; i<rows; i++)
15.             scanf("%d", &cost[i]);
16.         for(i=0; i<rows; i++) /* Cost computation of production and sales */
17.             for(j=0; j<cols; j++){
18.                 p[i][j] = production[i][j]*cost[j];
19.                 s[i][j] = sales[i][j]*cost[j];
20.             }
21.         weekly_production_sales(p, s);
22.         display_item_wise(p, s);
23.     }
```

You are required to write the code for below functions to accomplish the intended tasks. **Do not include any new variable(s) and avoid unnecessary redundancy in the code.** [6 + 6 + 3= 15M]

- a) Populating the arrays `p_week` and `s_week` and display the cost of weekly production and sales of items using the function “`void weekly_production_sales(int p[rows][cols], int s[rows][cols])`”. The output should be two separate values with respect to production and sales. [6]

- b) Compute the cost of item-wise production and sales to populate the arrays `product_wise` and `sale_wise` in the function “`void display_item_wise(int p[rows][cols], int s[rows][cols])`”. The output should be separate values with respect to the number of items. [6]
- c) In the context of sales and production, briefly write what should be printed when the below code is executed? [3]

```
pt = st = 0;
for(i=0; i<rows; i++){
    pt += p_week[i];
    st += s_week[i];
}
int d = st-pt;
printf("%d\n", pt);
printf("%d\n", st);
printf("%d\n", d);
```

3. You have to write a C function to rotate a linked list anti-clockwise by k elements, where k is a user input. You can see an example of linked list rotation in the sample run snapshot provided below. We will follow the following approach to rotate the linked list – We first make the linked list circular by making the last node point to the first node. Then we move $k-1$ steps forward from first node, and make k th node as head and make $(k-1)$ th node point to null. This completes the rotation by k .

Complete the implementation of the function `rotate()` making use of the code template given below. **Do not declare any new variables.** Assume that k is smaller than the number of nodes in the linked list, and all required header files are available to you. [12M]

```
struct LinkedList {
    int data;
    struct LinkedList *next;
};
```

```
typedef struct LinkedList node;
```

```
node *head; // ASSUME that “head” already points to the first node of list
```

```
void rotate(int k) {
```

```
    int i; //use this variable if required for any loops
```

```
    if (k == 0)
```

```
        return;
```

```
    node* current = head;
```

```
        // Your code goes here
```

```
}
```

```
pratik@Prithvi:~/code$ ./a.out
Enter the data at 1    10
Enter the data at 2    20
Enter the data at 3    30
Enter the data at 4    40
Enter the data at 5    50
[head] => 10 => 20 => 30 => 40 => 50 => [null]
Enter the value of k   3
After rotation by 3:
[head] => 40 => 50 => 10 => 20 => 30 => [null]
```

// The complete "driver code" is provided to you

```
int main() {  
    int k=0;  
    createList(5); //assume the implementation is given to you  
    printList();   //assume the implementation is given to you  
    printf("Enter the value of k  ");  
    scanf("%d", &k);  
    rotate(k);  
    printf("After rotation by %d:\n",k);  
    printList();  
    return 0;  
}
```

4. Find the output or the error (compile/run-time) for the following. No explanation required. [2*4= 8M]
(a) (b)

```
#include <stdio.h>  
void function(char**);  
int main()  
{  
    char *arr[] = { "ant", "bat", "cat", "dog", "bee", "fly" };  
    function(arr);  
    return 0;  
}  
void function(char **ptr)  
{  
    char *ptr1;  
    ptr1 = (ptr += sizeof(ptr))[-4];  
    printf("%s\n", ptr1);  
}
```

(c)

```
#include <stdio.h>  
int main()  
{    int x, y, z, i, j, k;  
    i=7;j=8;k=9;  
    z= i-7 && j++ < k;  
    x=y=z-=1;  
    ++x && ++y && ++z;  
    printf("x=%d,y=%d,z=%d\n",x,y,z);  
}
```

```
#include <stdio.h>  
void main()  
{  
    int arr[6] = {99,88,77,66,55,44};  
    int *p = arr+4;  
    for (int i=0;i<3;i++)  
        printf("%d", p[-i]);  
}
```

(d)

```
# include <stdio.h>  
int f(int a, int *pb, int **ppc)  
{  
    static int b, c;  
    **ppc += 1;  
    c = **ppc;  
    *pb += 2;  
    c = *pb;  
    a += 3;  
    return a + b + c;  
}  
  
void main()  
{  
    int c, *b, **a;  
    c = 4;  
    b = &c;  
    a = &b;  
    printf( "%d", f(c,b,a));  
}
```

5. Consider a structure, employee, containing name, salary and credit_point as member fields. An array of structure is used to store the overall details for a group of employees. The below program is meant to compute the appraisal for given record of employees. If the credit_point is set to 1, the respective employee is awarded with appraisal that increments his/her salary by 10%. On the contrary, if the credit_point is zero, the salary remains intact without any modification. These tasks are performed by a separate function "appraisal". The function appraisal is called for every employee, but only those employees who have the credit_point set will have their salaries modified. Once the appraisal task is completed for all employees, the employee names who are awarded appraisal along with their modified salaries are printed on the screen. *Without adding any new variable*, you are required to write the appropriate missing codes in the program so that its overall functioning can be achieved. Note that Missing Codes B and C need not be limited to one line. **[10M]**

```
1. #include<stdio.h>
2. #define cols 5
3. struct employee{
4.     char name[30];
5.     int salary;
6.     int credit_point;
7. } e[cols] = {"Andrew", 3000, 0},
8.             {"Mark", 2500, 1},
9.             {"Reinhard", 5000, 1},
10.            {"Adleman", 4000, 0},
11.            {"Pei", 3500, 1}};
12. struct employee appraisal(struct employee);
13. int main(){
14.     int i;
15.     printf("Employees awarded with appraisal and their modified salary\n");
16.     for(____MISSING CODE (A)____){
17.         ____MISSING CODE (B)____
18.     }
19.     return 0;
20. }
21. struct employee appraisal(struct employee e1){
22.     ____MISSING CODE (C)____
23. }
```

The End